

Stabilizing Native Low-Rank LLM Pretraining

Paul Janson^{1 2} Edouard Oyallon³ Eugene Belilovsky^{1 2}

Abstract

Foundation models have achieved remarkable success, yet their growing parameter counts pose significant computational and memory challenges. Low-rank factorization offers a promising route to reduce training and inference costs, but the community lacks a stable recipe for training models from scratch using exclusively low-rank weights while matching performance of the dense model. We demonstrate that Large Language Models (LLMs) can be trained from scratch using exclusively low-rank factorized weights for all non-embedding matrices without auxiliary “full-rank” guidance required by prior methods. While native low-rank training often suffers from instability and loss spikes, we identify uncontrolled growth in the spectral norm (largest singular value) of the weight matrix update as the dominant factor. To address this, we introduce **Spectron: Spectral** renormalization with orthogonalization, which dynamically bounds the resultant weight updates based on the current spectral norms of the factors. Our method enables stable, end-to-end factorized training with negligible overhead. Finally, we establish compute-optimal scaling laws for natively low-rank transformers, demonstrating predictable power-law behavior and improved inference efficiency relative to dense models. Our code is available at <https://github.com/Pauljanson002/spectron>

1. Introduction

Foundation models continue to scale in size, with model capacity remaining a primary driver of performance improvements in frontier systems (Kaplan et al., 2020; Hoffmann et al., 2022; Wei et al., 2022; Brown et al., 2020). Training these dense architectures, however, presents substantial

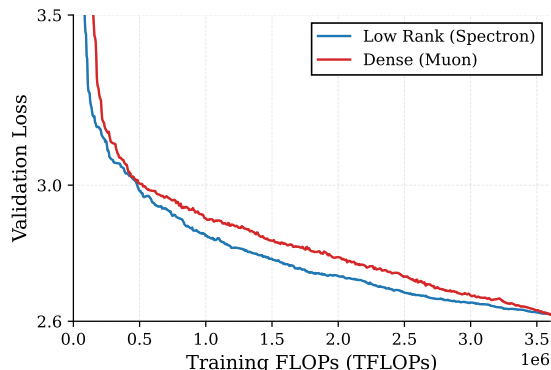


Figure 1. Natively Low-Rank Training Achieves Dense-Level Performance. Validation loss curves comparing a 780M dense Transformer (Vaswani et al., 2017) (red) against our 454M low-rank factorized Transformer (blue) across 3.5×10^6 training TFLOPs on FineWeb (Penedo et al., 2024). Our method **Spectron** enables stable end-to-end factorized training that matches dense performance at equal compute, yielding an inference-optimal model with substantially fewer parameters.

computational bottlenecks, particularly regarding memory constraints that motivate parallelization (Huh et al., 2024; Shazeer et al., 2017; Rajbhandari et al., 2020; Rivaud et al.; Nabli et al., 2025) and factorization (Khodak et al., 2021; Wei et al., 2024a; Chollet, 2017) strategies that distribute computation across accelerators while preserving throughput. Low-rank parameterization, where we express weight matrices as $W = AB^T$ with $A \in \mathbb{R}^{m \times r}$ and $B \in \mathbb{R}^{n \times r}$ for $r < \min(m, n)$ has achieved significant success in fine-tuning (Hu et al., 2022; Dettmers et al., 2023) by reducing both memory footprint and floating-point operations required for adapter-style updates. Yet practitioners have rarely applied this approach to pretraining large language models (LLMs), primarily because existing methods (Wei et al., 2024a; Wang et al., 2021; Huh et al., 2024) rely on workarounds that keep full-rank auxiliary weights, causing minimal adoption. A stable recipe for training factorized layers in modern transformer architectures (Touvron et al., 2023; Vaswani et al., 2017) has remained elusive.

Empirical evidence demonstrates that models tend to converge toward low-rank representations by the end of training (Martin & Mahoney, 2021; Galanti et al., 2025; Yang et al., 2023; Ramasinghe et al., 2025). Neural networks exhibit high compressibility (LeCun et al., 1989; Frankle & Carbin, 2019; Yu et al., 2017) and respond effectively to

¹Concordia University, Montreal, Canada ²Mila Quebec AI Institute, Montreal, Canada ³Sorbonne University, CNRS, Paris, France. Correspondence to: Paul Janson <paul.janson@mila.quebec>.

low-rank fine-tuning updates (Hu et al., 2022; Dettmers et al., 2023), suggesting that full-rank parameterization maintained throughout pretraining can be traded for low-rank representations to reduce the memory requirements. This observation motivates a fundamental question: *Can we train foundation models directly in a low-rank factorized regime from initialization while maintaining competitive performance?*

Answering this question carries both scientific and practical significance. Scientifically, exploiting low-rank structure during pretraining would provide insights into the learning dynamics of modern transformer architectures (Touvron et al., 2023; Vaswani et al., 2017) and reveal whether full-rank representations are necessary during optimization. Practically, native low-rank training could democratize foundation model development by reducing hardware requirements, potentially decreasing memory consumption by factors proportional to the rank reduction.

Training neural networks with native low-rank parameterization from scratch confronts severe instabilities when we factorize all non-embedding weight matrices to low rank. Unlike LoRA (Hu et al., 2022), which preserves frozen full-rank model weights, this approach encounters fundamental difficulties arising from how low-rank parameterization interacts with optimization dynamics. When we update factors independently, a given factorization $W = AB^\top$ permits infinitely many equivalent representations $W = (\lambda.A)(\frac{1}{\lambda}.B)$ for any $\lambda > 0$. This scaling invariance permits unbounded growth in λ , leading to unrestricted spectral norm (largest singular value) expansion, triggering exploding activations, and ultimately causing training divergence. The fundamental issue is that independent factor updates provide no mechanism to control the spectral properties of the resultant product matrix W .

Existing approaches acknowledge this difficulty by maintaining dependencies on full-rank components. Huh et al. (2024) apply low-rank constraints only to adapter modules while keeping backbone weights full-rank. GaLore (Zhao et al., 2024) and Chen et al. (2024) project gradients to low-rank subspaces but maintain full-rank weights during training, thus reducing only optimizer state memory. Wang et al. (2021) employ hybrid architectures by converting later layers to low rank while keeping initial layers full-rank, and they initialize from checkpoints of models pretrained with full-rank weights to avoid early instabilities. Most closely related, Wei et al. (2024a) restrict low-rank factorization to feed-forward layers and require full-rank initialization through self-guided training before transitioning to factorized weights. These methods avoid core instability by never fully committing to end-to-end factorized training.

We address this pathology through **Spectral renormalization** combined with **orthogonalization** that directly targets

the instability mechanism. Our method **Spectron** enables stable native low-rank training without auxiliary full-rank components by orthogonalizing gradient updates and constraining them to a region bounded by the inverse of the sum of spectral norms of the low-rank factors. We implement this constraint efficiently via power iteration-based (Vogels et al., 2019) spectral estimation and Newton Schulz (Jordan et al., 2024) based gradient orthogonalization that has negligible computational overhead, provably limiting spectral norm growth of the product matrix. This prevents the unbounded singular value expansion that destabilizes low-rank optimization.

We make the following contributions:

- **Spectral renormalization and orthogonalization:** We propose Spectron, an adaptive spectral norm constrained low-rank factor update with orthogonalization. We show that it bounds the resultant weight update spectral norms, enabling stable end-to-end factorized training from random initialization without auxiliary dense components (Section 4).
- **Empirical validation and scaling properties:** We demonstrate that our method applied to standard LLM pretraining with factorized non-embedding matrices enables stable training and achieves better final perplexity and downstream task accuracies compared to baselines. We show that factorized models trained with Spectron exhibit favorable scaling properties, matching or exceeding dense model performance (Figure 1) across three model scales on FineWeb (Penedo et al., 2024) pretraining (Section 5).
- **Compute optimal factorized transformers:** We derive compute-optimal scaling laws for low-rank transformers through isoFLOP analysis across 47M–1.5B parameters and 250M–90B tokens, establishing scaling relationships analogous to Chinchilla laws (Hoffmann et al., 2022) with exponents $N_{\text{opt}} \propto C^{0.479}$ and $D_{\text{opt}} \propto C^{0.521}$ (Section 6).

2. Related Works

Low-Rank Factorization in Neural Networks Deep neural networks are shown to inherently develop low-rank representations during training. Early work on network pruning demonstrated that low-rank structures can approximate trained models with minimal performance loss (LeCun et al., 1989; Yu et al., 2017), later attributed to implicit regularization in optimization dynamics (Martin & Mahoney, 2021; Yang et al., 2023; Frankle & Carbin, 2019). (Galanti et al., 2025) formalized this by proving that SGD with weight decay induces rank collapse in weight matrices. This observation motivated explicit low-rank parameterizations. Re-

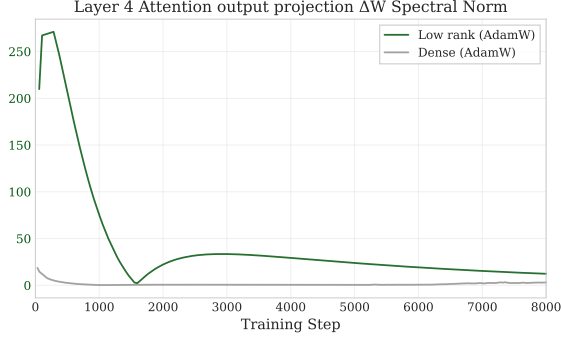


Figure 2. Low-Rank Parameterization Destabilizes Spectral Norm Dynamics. Weight update spectral norm ($\|\Delta W\|_2$) comparison between low-rank (green) and dense (gray) AdamW (Kingma & Ba, 2015) training on layer 4 attention output projection of a Transformer (Vaswani et al., 2017). Dense training maintains stable, bounded spectral norms, while low-rank factorization exhibits 10-30 $\times$ higher spectral norm magnitudes, revealing that the factorized updates (Equation (2)) fundamentally cause spectral instability.

searchers have explored factorized architectures in both convolutional and transformer models (Khodak et al., 2021; Wei et al., 2024b), while practitioners leverage low-rank structure to reduce fine-tuning costs through LoRA and its variants (Hu et al., 2022; Dettmers et al., 2023; Sharma et al., 2024). Applying low-rank methods to pretraining presents greater challenges. Huh et al. (2024) investigates LoRA-style adapters by freezing full-rank weights during training, while gradient compression methods exploit low-rank structure for memory efficiency (Zhao et al., 2024; Chen et al., 2024; Lialin et al., 2024). Hybrid approaches like (Wang et al., 2021) initialize from pretrained full-rank models to avoid optimization difficulties. Qiu et al. (2024) study factorized pretraining through the lens of μ P (Yang et al., 2022), deriving scaling laws for structured layers and showing that such layers can outperform dense layers at a fixed compute budget. Most closely related, Wei et al. (2024a) solely factorizes fully connected layers while maintaining auxiliary full-rank weights for optimization stability. Our method differs by converting all non-embedding matrices to low-rank parameterizations without auxiliary weights, eliminating memory overhead while directly addressing optimization challenges in native low-rank training.

Gradient Orthogonalization Gradient orthogonalization has emerged as a powerful technique for enhancing sample efficiency and optimization stability in deep neural networks (Jordan et al., 2024; Bernstein & Newhouse, 2024; Bernstein, 2025; Ma et al., 2024; Pethick et al., 2025). The Muon optimizer and its variants Jordan et al. (2024); Ahn et al. (2025); Si et al. (2025); Pethick et al. (2025) preprocess gradients through orthogonalization, achieving faster convergence and improved training dynamics. Recent work demonstrates that this approach scales effectively to large language model training (Liu et al., 2025a; Team et al., 2025). Several works (Kovalev, 2025; Bernstein, 2025;

Chen et al., 2025; Li & Hong, 2025; Fan et al., 2025) show that orthogonalized updates perform weight updates under spectral norm constraints. Wei et al. (2024a) observes uncontrolled spectral norm growth in weight matrices of the factors as a primary source of instability causing large gradient norms when training low-rank factorized networks. Yet they failed to see the effect on the resultant product matrix and opted to use dense guidance as the solution. We directly address this challenge by employing gradient orthogonalization and spectral renormalization to regulate spectral norm growth throughout training.

3. Background and Problem Formulation

We establish the foundational concepts underlying our approach and formalize the spectral instability challenge inherent to low-rank factorized training.

3.1. Background

Low-Rank Parameterization

We parameterize non-embedding layer weight matrices $W \in \mathbb{R}^{m \times n}$ of a transformer (Vaswani et al., 2017) neural network f_θ using low-rank factorizations to minimize computational overhead during training with next-token prediction cross-entropy loss $\mathcal{L}$. Specifically, we represent W as:

$$W = AB^\top, \quad A \in \mathbb{R}^{m \times r}, \quad B \in \mathbb{R}^{n \times r}, \quad (1)$$

where r denotes the rank with $r < \min(m, n)$.

During training, gradient-based updates are applied directly to the factors A and B . Via the chain rule, this leads to the composite weight update:

$$\Delta W = \Delta A B^\top + A \Delta B^\top + \Delta A \Delta B^\top \quad (2)$$

Operator Norms and Spectral Stability

We utilize the spectral norm $\|W\|_2$ to measure training stability, defined as the largest singular value of W :

$$\|W\|_2 = \sup_{x \in \mathbb{R}^n \setminus \{0\}} \frac{|Wx|_2}{|x|_2}, \quad (3)$$

where $\| \cdot \|_2$ denotes the Euclidean norm of the vector. The spectral norm satisfies the submultiplicative property

$$\|XY\|_2 \leq \|X\|_2 \|Y\|_2, \quad (4)$$

which proves essential for bounding composite updates in factorized layers. Following Bernstein (2025) and Yang et al. (2023), we employ the Root Mean Square (RMS) norm for a vector $y \in \mathbb{R}^m$:

$$|y|_{\text{rms}} = \sqrt{\frac{1}{m} \sum_{i=1}^m y_i^2}, \quad (5)$$

and the RMS-to-RMS operator norm for a matrix W , which measures the maximum amplification of entry-wise magnitudes:

$$\|W\|_{\text{rms} \rightarrow \text{rms}} = \sup_{x \in \mathbb{R}^n \setminus \{0\}} \frac{|Wx|_{\text{rms}}}{|x|_{\text{rms}}}. \quad (6)$$

This relationship enables us to control activation variance by constraining the spectral norm of weight updates.

Gradient Orthogonalization

Gradient orthogonalization has emerged as a principled technique for accelerating neural network training by constraining the geometry of hidden layer matrix parameter updates (Kovalev, 2025; Bernstein, 2025; Liu et al., 2025a; Jordan et al., 2024). The core principle normalizes all singular values of the update to unity.

Formally, given a gradient matrix $G_t = \nabla_{\theta} \mathcal{L}$ at time step t with singular value decomposition $G_t = U \Sigma V^{\top}$, we define the orthogonalization operation Ortho as:

$$O_t = \text{Ortho}(G_t) = UV^{\top}. \quad (7)$$

In practice, Jordan et al. (2024) orthogonalize the updates from SGD with momentum (M_t) using efficient Newton-Schulz iterations (Algorithm 2), yielding the update rule:

$$\theta_t \leftarrow \theta_{t-1} - \eta \cdot O_t, \quad (8)$$

where η denotes the learning rate.

3.2. The Spectral Instability Problem in Low-Rank Training

The optimization difficulties in low-rank training stem from uncontrolled growth in the spectral norm of the weight matrix updates. We note that this aligns with the observations of Wei et al. (2024a) where the authors found a correlation between large gradient norms and high spectral norms of the factors. Following the analysis of Bernstein (2025), consider the activation $y = Wx$ and its induced change under a weight update:

$$\Delta y = \Delta W x. \quad (9)$$

The change in activation can be bounded using the RMS-to-RMS operator norm:

$$|\Delta y|_{\text{rms}} \leq \|\Delta W\|_{\text{rms} \rightarrow \text{rms}} = \sqrt{\frac{n}{m}} \|\Delta W\|_2 \quad (10)$$

When the spectral norm $\|\Delta W\|_2$ grows excessively, the resulting activation changes destabilize training. Interestingly, this phenomenon is exclusive to low-rank factorized training and does not manifest in dense model training. Figure 2 demonstrates this contrast: dense training with AdamW (Kingma & Ba, 2015) maintains stable, bounded

Algorithm 1 Spectron

Require: Weight matrices $A \in \mathbb{R}^{m \times r}$, $B \in \mathbb{R}^{n \times r}$, step size $\eta > 0$

Require: Momentum decay $\beta \in [0, 1]$ (e.g. 0.9 or 0.95)

Require: Number of power iteration steps k_{power} (default: 1), Newton-Schulz iterations k_{ns} (default: 5)

```

1: Initialize  $u_A \in \mathbb{R}^m$  and  $u_B \in \mathbb{R}^n$  randomly and normalize:
    $u_A \leftarrow u_A / \|u_A\|$ ,  $u_B \leftarrow u_B / \|u_B\|$ 
2: Initialize momentum buffers  $M_A \leftarrow 0 \in \mathbb{R}^{m \times r}$ ,  $M_B \leftarrow 0 \in \mathbb{R}^{n \times r}$ 
3:  $t \leftarrow 1$ 
4: while not converged do
5:    $G_A^{(t)} \leftarrow \nabla_A \mathcal{L}$  ▷ Gradient of loss w.r.t.  $A$ 
6:    $G_B^{(t)} \leftarrow \nabla_B \mathcal{L}$  ▷ Gradient of loss w.r.t.  $B$ 
7:    $M_A \leftarrow \beta M_A + (1 - \beta) G_A^{(t)}$  ▷ Momentum update
8:    $M_B \leftarrow \beta M_B + (1 - \beta) G_B^{(t)}$  ▷ Momentum update
9:    $O_A^{(t)} \leftarrow \text{Ortho}(M_A, k_{\text{ns}})$  ▷ (Algorithm 2)
10:   $O_B^{(t)} \leftarrow \text{Ortho}(M_B, k_{\text{ns}})$  ▷ (Algorithm 2)
11:   $\sigma_A, u_A \leftarrow \text{PowerIter}(A^{(t)}, u_A, k_{\text{power}})$  ▷ (Algorithm 3)
12:   $\sigma_B, u_B \leftarrow \text{PowerIter}(B^{(t)}, u_B, k_{\text{power}})$  ▷ (Algorithm 3)
13:   $\Delta_A \leftarrow \frac{\eta}{\sigma_A + \sigma_B + 1} \cdot O_A^{(t)}$ 
14:   $\Delta_B \leftarrow \frac{\eta}{\sigma_A + \sigma_B + 1} \cdot O_B^{(t)}$ 
15:   $A^{(t)} \leftarrow A^{(t-1)} - \Delta_A$ 
16:   $B^{(t)} \leftarrow B^{(t-1)} - \Delta_B$ 
17:   $t \leftarrow t + 1$ 
18: return  $A^{(t)}, B^{(t)}$ 
    
```

spectral norms throughout optimization, while low-rank factorization exhibits $10\text{-}30\times$ higher spectral norm magnitudes. This reveals that the factorized update structure in Equation (2) fundamentally causes spectral instability.

4. Spectron: Spectral Renormalization and Orthogonalization

Drawing on recent works establishing orthogonalized updates as updates under a spectral norm constraint (Kovalev, 2025; Bernstein, 2025; Bernstein et al., 2020; Chen et al., 2025; Li & Hong, 2025; Fan et al., 2025), we propose to control training stability by bounding the spectral norm of the composite update:

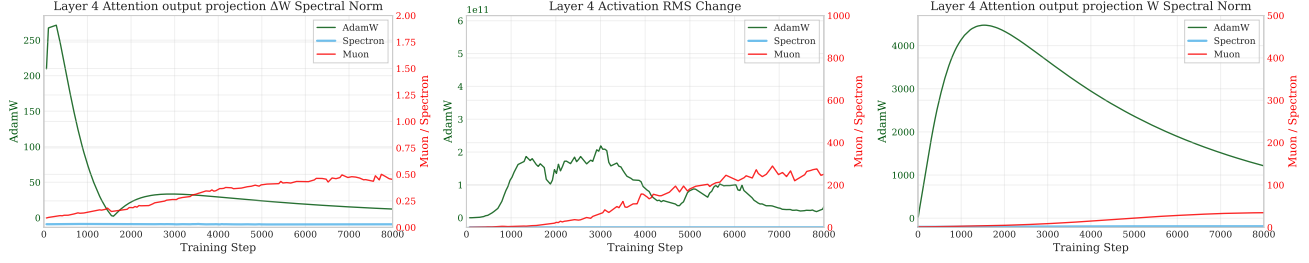
$$\|\Delta W\|_2 \leq \eta, \quad (11)$$

where η is a prescribed constraint radius defined by the learning rate. Our approach constrains the magnitudes of factor updates ΔA and ΔB such that the composite update ΔW maintains a stable spectral norm.

Bounding ΔW Through Adaptive Factor Constraints

We leverage gradient orthogonalization to ensure that factor updates remain within the local constraint radius defined by spectral norm (Kovalev, 2025):

$$\|\Delta A\|_2 \leq \rho, \quad \|\Delta B\|_2 \leq \rho \quad (12)$$



(a) Variation of $\|\Delta W\|_2$ with training step (b) Variation of $|\Delta y|_{rms}$ with training step (c) Variation of $\|W\|_2$ with training step

Figure 3. Spectral Norm Constraints Stabilize Low-Rank Training. Comparison of (a) weight update spectral norm $\|\Delta W\|_2$, (b) activation RMS change $|\Delta y|_{rms}$, and (c) weight spectral norm $\|W\|_2$ across 8000 training steps for layer 4 attention output projection of a 94M parameter Factorized Transformer (Vaswani et al., 2017). AdamW (Kingma & Ba, 2015) (green, left axis) exhibits explosive growth in all metrics with unconstrained spectral norm dynamics. Muon (Jordan et al., 2024) (red, right axis) achieves moderate control through gradient orthogonalization (Bernstein & Newhouse, 2024). Our method, Spectron (blue, right axis) maintains bounded spectral norms throughout training by adaptively constraining factor updates, demonstrating stable optimization. Note that AdamW curves use a different y-axis scale (left) compared to Muon and Spectron (right) for visualization purposes.

for a constraint radius ρ to be determined adaptively. Applying the triangle inequality and submultiplicativity of the spectral norm to Equation (2):

$$\|\Delta W\|_2 \leq \|\Delta A B^\top\|_2 + \|A \Delta B^\top\|_2 + \|\Delta A \Delta B^\top\|_2. \quad (13)$$

Using property (4) and bounds in Equation (12):

$$\|\Delta W\|_2 \leq \rho \|B\|_2 + \rho \|A\|_2 + \rho^2. \quad (14)$$

For typical learning rates where $\rho < 1$, we obtain the upper bound:

$$\|\Delta W\|_2 \leq \rho (\|A\|_2 + \|B\|_2 + 1). \quad (15)$$

To satisfy the global constraint $\|\Delta W\|_2 \leq \eta$, we dynamically set the local constraint radius as:

$$\rho = \frac{\eta}{\|A\|_2 + \|B\|_2 + 1}. \quad (16)$$

This adaptive scaling ensures that orthogonalized updates $(\Delta A, \Delta B)$ satisfying Equation (16) induce a composite update ΔW that respects the required spectral norm bound, regardless of the current magnitudes of A and B . We estimate $\|A\|_2$ and $\|B\|_2$ efficiently using a single power iteration (Algorithm 3) and orthogonalize using five Newton Schulz iterations (Algorithm 2), following recent works on efficient spectral norm approximation (Ahn et al., 2025; Vogels et al., 2019) and orthogonalization (Jordan et al., 2024). In practice, Spectron can be implemented by combining orthogonalized updates from Muon (Jordan et al., 2024) for each factor with an explicit spectral renormalization step based on the estimated norms of both factors. The complete algorithm of **Spectron** is presented in Algorithm 1.

Empirical Validation of Spectral Constraints

Figure 3 demonstrates the effectiveness of our spectral renormalization approach across 8000 training steps for

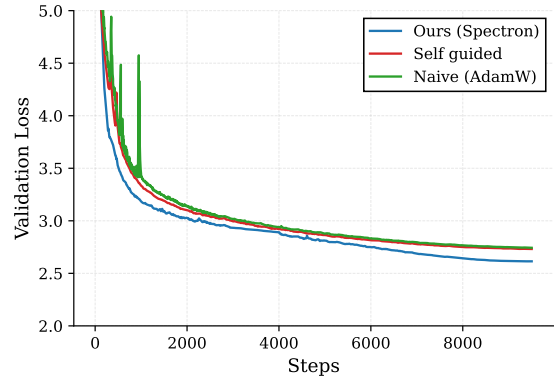


Figure 4. Spectrally Normalized Low-Rank Training Outperforms Baselines. Validation loss on FineWeb (Penedo et al., 2024) held-out set during Factorized Transformer-M (297M) pretraining comparing Spectron (blue), self-guided training (red), and naive AdamW (green). Our approach achieves both faster initial convergence and superior final performance (Table 1), outperforming self-guided training despite its dense auxiliary full rank weights, while maintaining sub-1% computational overhead compared to self-guided’s 25% additional FLOPs.

layer 4 attention output projection of a 94M parameter factorized Transformer (Vaswani et al., 2017). Standard AdamW (Kingma & Ba, 2015) exhibits explosive growth in weight update spectral norm $\|\Delta W\|_2$, activation RMS change $|\Delta y|_{rms}$, and weight spectral norm $\|W\|_2$, confirming the instability identified in Section 3. The orthogonalized optimizer Muon (Jordan et al., 2024) achieves moderate control through gradient orthogonalization alone (Bernstein & Newhouse, 2024). Our method maintains bounded spectral norms throughout training by adaptively constraining factor updates according to Equation (16), demonstrating stable optimization dynamics across all tracked metrics.

Method	Perplexity (↓)	HellaSwag (↑)	PIQA (↑)	Arc Easy (↑)
Factorized Llama-94M				
Naive (AdamW) (Kingma & Ba, 2015)	26.43	26.83	57.83	31.44
Self guided (Wei et al., 2024a)	24.17	26.62	58.32	30.93
Ours (Spectron)	21.86	27.52	58.60	31.90
Factorized Llama-297M				
Naive (AdamW) (Kingma & Ba, 2015)	15.54	31.59	63.00	34.09
Self guided (Wei et al., 2024a)	15.53	31.08	62.89	34.05
Ours (Spectron)	14.62	33.98	63.55	35.10
Factorized Llama-454M				
Naive (AdamW) (Kingma & Ba, 2015)	14.57	34.05	65.18	35.98
Self guided (Wei et al., 2024a)	13.70	34.85	64.91	35.82
Ours (Spectron)	12.11	40.11	66.76	36.78

Table 1. **Comparative Performance of Low-Rank Factorized Training Methods Across Model Scales.** Perplexity (↓), and downstream task accuracies (HellaSwag (Zellers et al., 2019) (↑), PIQA (Bisk et al., 2020) (↑), Arc Easy (Clark et al., 2018) (↑)) for three factorized Transformer variants (94M, 297M, 454M parameters) trained with naive AdamW (Kingma & Ba, 2015), self-guided training (Wei et al., 2024a), and our method Spectron. Our approach consistently achieves superior performance across all metrics and model sizes.

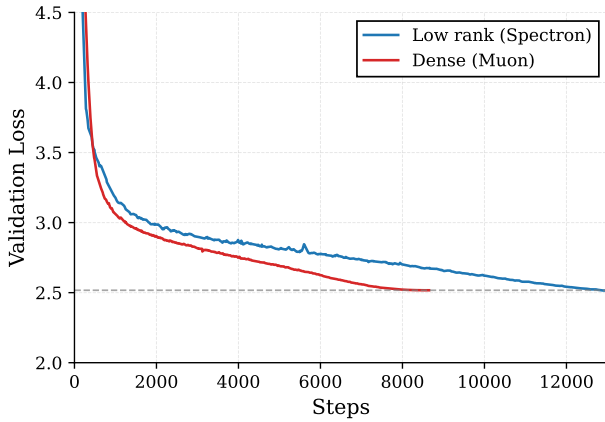


Figure 5. **Low-Rank Factorization Matches Dense Performance with Longer Training.** Validation loss comparison between Dense Transformer-L (780M parameters) and our Low-Rank Factorized Transformer-L (454M parameters) trained for **equal FLOPs** by matching training steps. Despite a $\sim 42\%$ parameter reduction, our factorized model (blue) converges to the same final validation loss as the dense baseline (red), demonstrating that compute-equivalent training yields an inference-optimal model.

5. Experiments

Experimental Setup We evaluate Spectron on LLaMA-style transformer architectures (Touvron et al., 2023) across multiple scales. We train three full-rank variants: Llama-134M, Llama-500M and Llama-780M, alongside corresponding factorized versions with rank ratio 0.25 ($r = 0.25n$): Factorized Llama-94M, Factorized Llama-297M, and Factorized Llama-454M. All non-embedding weight matrices use low-rank decompositions in factorized models (implementation details in Appendix E).

All models are pretrained on FineWeb (Penedo et al., 2024) with a 100M token validation set. Dense baselines train to Chinchilla (Hoffmann et al., 2022)-optimal token counts and corresponding factorized models train for matched FLOPs. We train the dense baselines with Muon (Jordan et al., 2024) optimizer for fair comparison. We report validation perplexity, and normalized accuracy on HellaSwag (Zellers et al., 2019), PIQA (Bisk et al., 2020), and ARC-easy (Clark et al., 2018) via lm-evaluation-harness (Gao et al., 2023).

Baselines. We compare against self-guided training (Wei et al., 2024a), the current state-of-the-art for stable low-rank pretraining. This method supervises low-rank parameters with concurrent dense weight updates during the first half of training: While effective, this incurs $\sim 25\%$ FLOP overhead during guidance (Appendix C). We additionally benchmark naive AdamW (Kingma & Ba, 2015) training.

Our method introduces minimal overhead: Newton-Schulz orthogonalization (Jordan et al., 2024) (Algorithm 2) adds $6k_{\text{ns}}nm^2$ FLOPs ($<1\%$ for typical architectures), while power iteration spectral norm estimation (Algorithm 3) requires only $2mn$ FLOPs per matrix of size $m \times n$. Total overhead remains sub-1%—a $25\times$ reduction versus self-guided training.

5.1. Comparison to Low rank training baselines

Figure 4 demonstrates that spectron enables both faster convergence and superior final performance compared to existing low-rank training methods. Our approach achieves stable training at higher learning rates that cause baseline

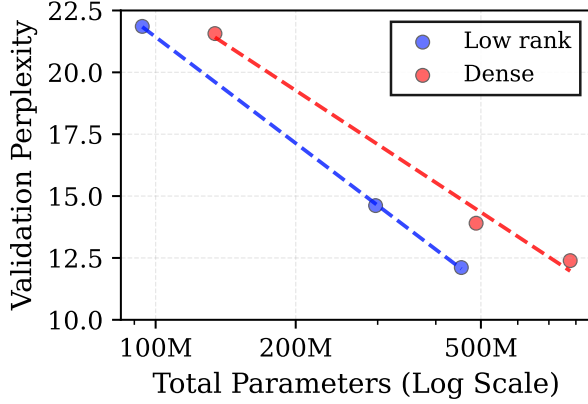


Figure 6. Low-Rank Factorization Improves Scaling Efficiency. Validation perplexity(↓) comparison between low-rank factorized models (blue) and dense models (red) across model sizes from 100M to 780M parameters. Low-rank models achieve consistently lower perplexity across scales and need a lower parameter count for a given perplexity, showing inference efficiency

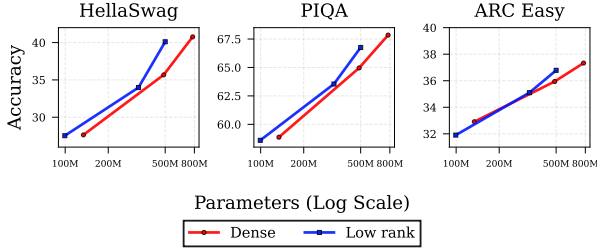


Figure 7. Low-Rank Models Achieve Superior Downstream Performance with Fewer Parameters: Accuracy(↑) comparison across three benchmark tasks (HellaSwag (Zellers et al., 2019), PIQA (Bisk et al., 2020), ARC Easy (Clark et al., 2018)) for dense versus low-rank models trained with equal computational budgets. Low-rank architectures (blue) consistently outperform dense baselines (red) across all model scales, demonstrating strong downstream performance under reduced inference cost

divergence (Appendix B.3), indicating effective constraint of spectral norm dynamics. Table 1 confirms consistent improvements across model scales: Spectron reduces perplexity by 6–12% versus self-guided training and 6–17% versus naive AdamW, with corresponding downstream accuracy gains.

5.2. Comparison to Dense Model Training

To evaluate whether low-rank factorization inherently limits model capacity, we compare factorized transformers against dense baselines trained with equal computational budgets.

Figure 1 and 5 shows that when trained for equal FLOPs, Factorized Transformer-L (454M) converges to the same validation loss as Dense Transformer-L (780M), demonstrating that low-rank capacity limitations can be overcome

through extended training under stable optimization (Sardana et al., 2024). Even with 42% reduced parameters, we recover the same performance, yielding inference savings by the same amount. This phenomenon strengthens at scale: Figure 6 reveals that factorized models achieve consistently lower perplexity than parameter-matched dense baselines. This yields a more compact model for a given perplexity threshold, thereby substantially reducing inference costs.

Figure 7 extends these findings to downstream evaluation, where factorized transformers match or exceed dense performance across HellaSwag (Zellers et al., 2019), PIQA (Bisk et al., 2020), and ARC Easy (Clark et al., 2018) benchmarks.

6. Towards compute optimal Low-Rank Pretraining

Having established performance parity with dense models under equal compute (Section 5), we investigate the fundamental scaling properties of low-rank architectures: *given a fixed computational budget, what is the optimal allocation between model parameters and training tokens?*

We adopt the IsoFLOP profiling approach from Hoffmann et al. (2022), training factorized transformers ranging from 47M to 1.5B parameters across four compute budgets (2.20×10^{18} to 3.57×10^{19} FLOPs). Token budgets are adjusted inversely to maintain constant FLOPs per configuration, with quadratic fits to each IsoFLOP curve identifying the loss-minimizing model size at each compute level.

Figure 9 demonstrates through 39 extensive pretraining runs that low-rank pretraining exhibits well-defined compute-optimal model sizes, with clear loss minima at each budget level. These optima shift rightward with increased compute, mirroring the scaling structure of dense transformers (Hoffmann et al., 2022) and confirming that low-rank architectures follow predictable optimization frontiers. Figure 8 reveals the precise relationships: optimal model size scales as $N_{\text{opt}} \propto C^{0.479}$ (versus Chinchilla’s 0.49), while training tokens scale as $D_{\text{opt}} \propto C^{0.521}$ (versus 0.51). This modest deviation indicates that factorized architectures achieve compute-optimality at smaller model sizes, yielding inference-efficient models that compensate through extended training (Sardana et al., 2024).

Low-rank pretraining naturally produces models that are simultaneously compute-optimal and inference-efficient. For a given compute budget, the resulting architecture contains fewer parameters than a comparably-trained dense model while maintaining equivalent performance (Section 5). This property is particularly advantageous when inference efficiency is prioritized in settings similar to what Sardana et al. (2024) explores, where data availability exceeds compute constraints, enabling full utilization of a large corpora. A parametric study of this using Approach 3 of Hoffmann et al.

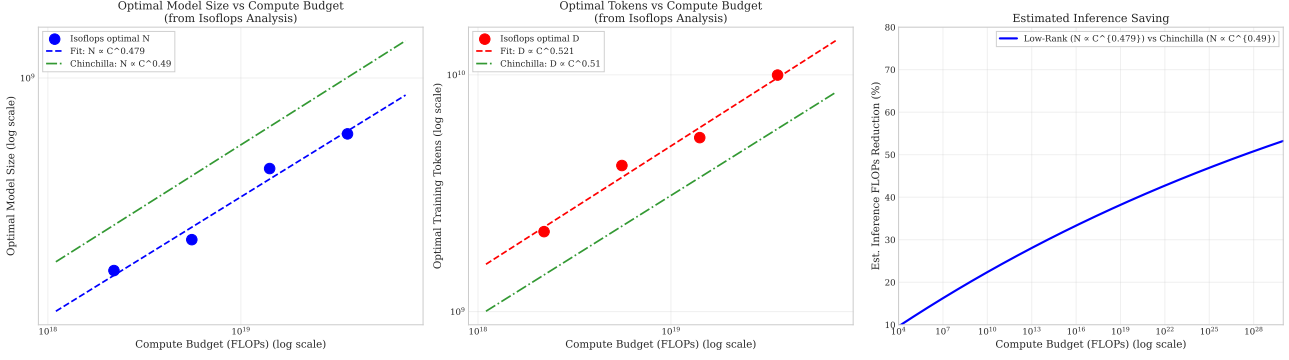


Figure 8. Factorized Models Scale More Conservatively Than Dense Transformers, Yielding Substantial Inference Efficiency Gains. (Left) Optimal model size versus compute budget for low-rank architectures follows $N_{\text{opt}} \propto C^{0.479}$ (blue points), compared to Chinchilla’s $N \propto C^{0.49}$ (green dashed reference). (Center) Optimal training tokens scale as $D_{\text{opt}} \propto C^{0.521}$ (red points), versus Chinchilla’s $D \propto C^{0.51}$. The reduced parameter scaling exponent (0.479 vs. 0.49) indicates that compute-optimal low-rank models are smaller than their dense counterparts at equivalent training budgets, requiring proportionally more training tokens. (Right) Estimated inference cost savings, computed as $(1 - N_{\text{opt}}/N_{\text{Chinchilla}}) \times 100 = (1 - 1/C^{0.011}) \times 100\%$, assuming inference cost scales as $2N_{\text{opt}} \cdot D_{\text{inf}}$ with identical proportionality constants for both low-rank and dense models. Under contemporary FLOP budgets for training ($\sim 10^{26}$ FLOPs), low-rank models achieve up to 50% inference cost reduction compared to Chinchilla-optimal dense transformers. This trade-off favors scenarios where inference efficiency and compact deployment are prioritized over token budget.

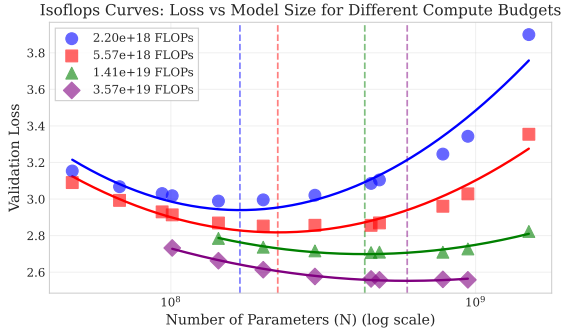


Figure 9. Low-Rank Architectures Exhibit Clear Compute-Optimal Model Sizes. Validation loss versus parameter count for factorized transformers trained at four compute budgets (2.20e+18 to 3.57e+19 FLOPs). Each IsoFLOP curve displays a distinct minimum (vertical dashed lines), with optimal model size increasing monotonically with compute budget. This replicates the fundamental structure of Chinchilla scaling laws (Hoffmann et al., 2022) in the low-rank regime, establishing that compute-optimal training requires balanced scaling of both model size and training duration.

(2022) appears in Appendix D.

The near-equivalence of scaling exponents (0.479 vs. 0.49 for parameters; 0.521 vs. 0.51 for tokens) suggests that low-rank factorization does not fundamentally alter transformer scaling dynamics but rather shifts the compute-optimal frontier toward smaller, more token-intensive configurations. But this is non-trivial at scale, as described in Figure 8 (right), where at modern compute budgets this can be a significant reduction in the inference cost estimated by $2N_{\text{opt}}D_{\text{inf}}$, where D_{inf} denotes total inference tokens assumed to be the same for both configurations.

7. Conclusion

In this paper, we present Spectron, a method for training large language models (LLMs) from scratch using exclusively low-rank factorized weight matrices, eliminating the need for auxiliary full-rank weights. Our core contribution identifies unbounded spectral norm growth as the fundamental source of training instability in factorized architectures. We address this through a combination of spectral renormalization and gradient orthogonalization that adaptively constrains weight updates based on the spectral norm of factors. This approach bounds composite weight update norms while introducing negligible computational overhead relative to existing methods (Wei et al., 2024a).

Our empirical validation demonstrates that factorized transformers achieve performance parity with dense baselines under equivalent FLOP budgets, despite significantly fewer parameters. Through systematic IsoFLOP analysis, we derive compute-optimal scaling laws specific to native low-rank pretraining. The resulting relationships yield optimal model size scaling as $N_{\text{opt}} \propto C^{0.479}$ and training token requirements as $D_{\text{opt}} \propto C^{0.521}$, revealing that compute-optimal factorized architectures favor smaller model configurations trained on proportionally larger datasets. This scaling behavior translates directly to substantial inference-time compute savings. Future works could develop communication strategies specifically tailored for factorized architectures to reduce distributed training overhead. We primarily focus on large language models; the underlying principles suggest natural extensions to multimodal architectures. Finally, we anticipate that native low-rank training could enable more flexible pretraining paradigms.

Limitations. Our experiments are conducted on models up to 1.5B parameters which remain substantially smaller than contemporary foundation model pretraining regimes that often operate at trillion-token scale. Consequently, while the empirical results demonstrate consistent trends within the studied regime, direct validation at frontier scales remains an open question. Nevertheless, the fitted scaling laws suggest that the observed behavior may continue to hold at larger scales, providing preliminary evidence for the broader applicability of our conclusions. In addition, we observe wall-clock time gap between dense and low-rank training in our current implementation. Dense training benefits from highly optimized fused kernels, whereas our current low-rank implementation relies on two unfused matrix multiplications whose kernel launch overhead dominates at the scales considered here. We believe that custom fused kernels can substantially reduce these inefficiencies.

Acknowledgement

EB and PJ acknowledge funding from FRQNT and NSERC. EO acknowledges funding from PEPR IA (grant SHARP ANR-23-PEIA-0008). We acknowledge compute resources from IDRIS under the allocation 2025-AD011015884R1 and Digital Research Alliance of Canada.

Impact statement

This work enables training large language models entirely with low-rank factorized weights, producing models that match dense performance at equal compute while using fewer parameters. The main benefit is efficiency: lower memory, inference, and energy costs can reduce the financial and environmental burden of deploying foundation models and broaden access for those with limited hardware. As with any advance in efficient model training, these gains are dual-use, since cheaper training and inference also lower the barrier to misuse. Our contribution is methodological and introduces no capabilities beyond those of comparable dense models, so the associated risks are inherited from large language models in general rather than created here. We believe the efficiency and accessibility benefits outweigh these marginal risks.

References

- Ahn, K., Xu, B., Abreu, N., Fan, Y., Magakyan, G., Sharma, P., Zhan, Z., and Langford, J. Dion: Distributed orthonormalized updates. *arXiv preprint arXiv:2504.05295*, 2025.
- Bernstein, J. Deriving muon, 2025. URL <https://jeremybernste.in/writing/deriving-muon>.
- Bernstein, J. and Newhouse, L. Old optimizer, new norm: An anthology. *arXiv preprint arXiv:2409.20325*, 2024.
- Bernstein, J., Vahdat, A., Yue, Y., and Liu, M.-Y. On the distance between two neural networks and the stability of learning. *Advances in Neural Information Processing Systems*, 33:21370–21381, 2020.
- Bisk, Y., Zellers, R., Gao, J., Choi, Y., et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pp. 7432–7439, 2020.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.
- Chen, L., Li, J., and Liu, Q. Muon optimizes under spectral norm constraints. *arXiv preprint arXiv:2506.15054*, 2025.
- Chen, X., Feng, K., Li, C., Lai, X., Yue, X., Yuan, Y., and Wang, G. Fira: Can we achieve full-rank training of llms under low-rank constraint? *arXiv preprint arXiv:2410.01623*, 2024.
- Chollet, F. Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 1251–1258, 2017.
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems*, 36:10088–10115, 2023.
- Elfwing, S., Uchibe, E., and Doya, K. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural networks*, 107:3–11, 2018.
- Fan, C., Schmidt, M., and Thrampoulidis, C. Implicit bias of spectral descent and muon on multiclass separable data. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=Zn2ajV1kTQ>.
- Frankle, J. and Carbin, M. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *International Conference on Learning Representations*, 2019.

- Galanti, T., Siegel, Z. S., Gupte, A., and Poggio, T. A. Sgd with weight decay secretly minimizes the ranks of your neural networks. In Chen, B., Liu, S., Pilanci, M., Su, W., Sulam, J., Wang, Y., and Zhu, Z. (eds.), *Conference on Parsimony and Learning*, volume 280 of *Proceedings of Machine Learning Research*, pp. 1388–1412. PMLR, 24–27 Mar 2025. URL <https://proceedings.mlr.press/v280/galanti25a.html>.
- Gao, L., Tow, J., Abbasi, B., Biderman, S., Black, S., DiPofi, A., Foster, C., Golding, L., Hsu, J., Le Noac’h, A., Li, H., McDonnell, K., Muennighoff, N., Ociepa, C., Phang, J., Reynolds, L., Schoelkopf, H., Skowron, A., Sutawika, L., Tang, E., Thite, A., Wang, B., Wang, K., and Zou, A. A framework for few-shot language model evaluation, 12 2023. URL <https://zenodo.org/records/10256836>.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Vinyals, O., Rae, J. W., and Sifre, L. An empirical analysis of compute-optimal large language model training. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=iBBcRUlOAPR>.
- Hu, E. J., yelong shen, Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- Huber, P. J. Robust estimation of a location parameter. In *Breakthroughs in statistics: Methodology and distribution*, pp. 492–518. Springer, 1992.
- Huh, M., Cheung, B., Bernstein, J., Isola, P., and Agrawal, P. Training neural networks from scratch with parallel low-rank adapters. *arXiv preprint arXiv:2402.16828*, 2024.
- Jordan, K., Jin, Y., Boza, V., Jiacheng, Y., Cesista, F., Newhouse, L., and Bernstein, J. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon/>.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Khodak, M., Tenenholz, N. A., Mackey, L., and Fusi, N. Initialization and regularization of factorized neural layers. In *International Conference on Learning Representations*, 2021.
- Kingma, D. P. and Ba, J. L. Adam: A method for stochastic gradient descent. In *ICLR: international conference on learning representations*, pp. 1–15, 2015.
- Kovalev, D. Understanding gradient orthogonalization for deep learning via non-euclidean trust-region optimization. *arXiv preprint arXiv:2503.12645*, 2025.
- LeCun, Y., Denker, J., and Solla, S. Optimal brain damage. *Advances in neural information processing systems*, 2, 1989.
- Li, J. and Hong, M. A note on the convergence of muon. *arXiv preprint arXiv:2502.02900*, 2025.
- Lialin, V., Muckatira, S., Shivagunde, N., and Rumshisky, A. Relora: High-rank training through low-rank updates. In *International Conference on Learning Representations*, volume 2024, pp. 49405–49421, 2024.
- Liu, J., Su, J., Yao, X., Jiang, Z., Lai, G., Du, Y., Qin, Y., Xu, W., Lu, E., Yan, J., et al. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025a.
- Liu, Z., Zhang, R., Wang, Z., Yan, M., Yang, Z., Hovland, P. D., Nicolae, B., Cappello, F., Tang, S., and Zhang, Z. Cola: Compute-efficient pre-training of llms via low-rank activation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 4627–4645, 2025b.
- Loshchilov, I. and Hutter, F. Sgdr: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983*, 2016.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- Ma, C., Gong, W., Scetbon, M., and Meeds, E. Swan: Sgd with normalization and whitening enables stateless llm training. *arXiv preprint arXiv:2412.13148*, 2024.
- Martin, C. H. and Mahoney, M. W. Implicit self-regularization in deep neural networks: Evidence from random matrix theory and implications for learning. *Journal of Machine Learning Research*, 22(165):1–73, 2021.
- Mo, Z., Huang, L.-K., and Pan, S. J. Parameter and memory efficient pretraining via low-rank riemannian optimization. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=i0zzO7Hslk>.
- Nabli, A., Fournier, L., ERBACHER, P., Serrano, L., Belilovsky, E., and Oyallon, E. ACCO: Accumulate while you communicate for communication-overlapped

- sharded LLM training. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=lqKUVyymXs>.
- Nair, V. and Hinton, G. E. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th international conference on machine learning (ICML-10)*, pp. 807–814, 2010.
- Nocedal, J. Updating quasi-newton matrices with limited storage. *Mathematics of computation*, 35(151):773–782, 1980.
- Penedo, G., Kydlíček, H., Lozhkov, A., Mitchell, M., Raffel, C. A., Von Werra, L., Wolf, T., et al. The fineweb datasets: Decanting the web for the finest text data at scale. *Advances in Neural Information Processing Systems*, 37: 30811–30849, 2024.
- Pethick, T., Xie, W., Antonakopoulos, K., Zhu, Z., Silveti-Falls, A., and Cevher, V. Training deep learning models with norm-constrained lmos. In *International Conference on Machine Learning*, pp. 49069–49104. PMLR, 2025.
- Qiu, S., Potapczynski, A., Finzi, M., Goldblum, M., and Wilson, A. G. Compute better spent: replacing dense layers with structured matrices. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 41698–41716, 2024.
- Rajbhandari, S., Rasley, J., Ruwase, O., and He, Y. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–16. IEEE, 2020.
- Ramasinghe, S., Ajanthan, T., Avraham, G., Zuo, Y., and Long, A. Subspace networks: Scaling decentralized training with communication-efficient model parallelism. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=kke9TwtKi0>.
- Rivaud, S., Fournier, L., Pumir, T., Belilovsky, E., Eickenberg, M., and Oyallon, E. Petra: Parallel end-to-end training with reversible architectures. In *The Thirteenth International Conference on Learning Representations*.
- Sardana, N., Portes, J., Doubov, S., and Frankle, J. Beyond chinchilla-optimal: accounting for inference in language model scaling laws. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 43445–43460, 2024.
- Sharma, P., Ash, J. T., and Misra, D. The truth is in there: Improving reasoning in language models with layer-selective rank reduction. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=ozX92bu8VA>.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=BlckMDqlg>.
- Si, C., Zhang, D., and Shen, W. Adamuon: Adaptive muon optimizer. *arXiv preprint arXiv:2507.11005*, 2025.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- Team, K., Bai, Y., Bao, Y., Chen, G., Chen, J., Chen, N., Chen, R., Chen, Y., Chen, Y., Chen, Y., Chen, Z., Cui, J., Ding, H., Dong, M., Du, A., Du, C., Du, D., Du, Y., Fan, Y., Feng, Y., Fu, K., Gao, B., Gao, H., Gao, P., Gao, T., Gu, X., Guan, L., Guo, H., Guo, J., Hu, H., Hao, X., He, T., He, W., He, W., Hong, C., Hu, Y., Hu, Z., Huang, W., Huang, Z., Huang, Z., Jiang, T., Jiang, Z., Jin, X., Kang, Y., Lai, G., Li, C., Li, F., Li, H., Li, M., Li, W., Li, Y., Li, Y., Li, Z., Li, Z., Lin, H., Lin, X., Lin, Z., Liu, C., Liu, C., Liu, H., Liu, J., Liu, J., Liu, L., Liu, S., Liu, T. Y., Liu, T., Liu, W., Liu, Y., Liu, Y., Liu, Y., Liu, Y., Liu, Z., Lu, E., Lu, L., Ma, S., Ma, X., Ma, Y., Mao, S., Mei, J., Men, X., Miao, Y., Pan, S., Peng, Y., Qin, R., Qu, B., Shang, Z., Shi, L., Shi, S., Song, F., Su, J., Su, Z., Sun, X., Sung, F., Tang, H., Tao, J., Teng, Q., Wang, C., Wang, D., Wang, F., Wang, H., Wang, J., Wang, J., Wang, J., Wang, S., Wang, S., Wang, Y., Wang, Y., Wang, Y., Wang, Y., Wang, Y., Wang, Z., Wang, Z., Wang, Z., Wei, C., Wei, Q., Wu, W., Wu, X., Wu, Y., Xiao, C., Xie, X., Xiong, W., Xu, B., Xu, J., Xu, J., Xu, L. H., Xu, L., Xu, S., Xu, W., Xu, X., Xu, Y., Xu, Z., Yan, J., Yan, Y., Yang, X., Yang, Y., Yang, Z., Yang, Z., Yang, Z., Yao, H., Yao, X., Ye, W., Ye, Z., Yin, B., Yu, L., Yuan, E., Yuan, H., Yuan, M., Zhan, H., Zhang, D., Zhang, H., Zhang, W., Zhang, X., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Z., Zhao, H., Zhao, Y., Zheng, H., Zheng, S., Zhou, J., Zhou, X., Zhou, Z., Zhu, Z., Zhuang, W., and Zu, X. Kimi k2: Open agentic intelligence, 2025. URL <https://arxiv.org/abs/2507.20534>.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. At-

- tention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., Carey, C. J., Polat, İ., Feng, Y., Moore, E. W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E. A., Harris, C. R., Archibald, A. M., Ribeiro, A. H., Pedregosa, F., van Mulbregt, P., and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.
- Vogels, T., Karimireddy, S. P., and Jaggi, M. Powersgd: Practical low-rank gradient compression for distributed optimization. *Advances in Neural Information Processing Systems*, 32, 2019.
- Wang, H., Agarwal, S., and Papailiopoulos, D. Pufferfish: Communication-efficient models at no extra cost. *Proceedings of Machine Learning and Systems*, 3:365–386, 2021.
- Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., Chi, E. H., Hashimoto, T., Vinyals, O., Liang, P., Dean, J., and Fedus, W. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022. ISSN 2835-8856. URL <https://openreview.net/forum?id=yzkSU5zdwD>. Survey Certification.
- Wei, X., Moalla, S., Pascanu, R., and Gulcehre, C. Building on efficient foundations: Effective training of LLMs with structured feedforward layers. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024a. URL <https://openreview.net/forum?id=WxLVYZbIew>.
- Wei, X., Moalla, S., Pascanu, R., and Gulcehre, C. Investigating low-rank training in transformer language models: Efficiency and scaling analysis. *arXiv preprint arXiv:2407.09835*, 2024b.
- Yang, G., Hu, E. J., Babuschkin, I., Sidor, S., Liu, X., Farhi, D., Ryder, N., Pachocki, J., Chen, W., and Gao, J. Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer, March 2022. URL <http://arxiv.org/abs/2203.03466>. arXiv:2203.03466 [cond-mat].
- Yang, G., Simon, J. B., and Bernstein, J. A spectral condition for feature learning. *arXiv preprint arXiv:2310.17813*, 2023.
- Yu, X., Liu, T., Wang, X., and Tao, D. On compressing deep models by low rank and sparse decomposition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 7370–7379, 2017.
- Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 4791–4800, 2019.
- Zhao, J., Zhang, Z., Chen, B., Wang, Z., Anandkumar, A., and Tian, Y. Galore: Memory-efficient llm training by gradient low-rank projection. In *International Conference on Machine Learning*, pp. 61121–61143. PMLR, 2024.
- Zhao, J., Zhang, Y., Li, X., Liu, H., and Cannistraci, C. V. Sparse spectral training and inference on euclidean and hyperbolic neural networks. In *International Conference on Machine Learning*, pp. 77976–78002. PMLR, 2025.

A. Algorithms

We present two auxiliary algorithms utilized in our method. Algorithm 2 describes the Newton-Schulz iteration (Jordan et al., 2024), an efficient procedure for matrix orthogonalization. Algorithm 3 presents the power iteration method (Vogels et al., 2019), which we use to estimate spectral norms of the low-rank factors.

Algorithm 2 Newton-Schulz Orthogonalization

Require: Gradient/Momentum matrix $G \in \mathbb{R}^{m \times n}$, iteration steps $k_{ns} = 5$, numerical stability constant $\epsilon = 10^{-7}$
Require: Newton-Schulz coefficients: $(a, b, c) = (3.4445, -4.7750, 2.0315)$
Ensure: Orthogonalized matrix $X_{k_{ns}} \in \mathbb{R}^{m \times n}$

- 1: $X_0 \leftarrow G$ ▷ Initialize with input matrix
- 2: $s \leftarrow \frac{1}{\|X_0\|_F + \epsilon}$ ▷ Frobenius normalization factor
- 3: $X_1 \leftarrow sX_0$ ▷ Normalized initialization
- 4: **if** $m > n$ **then** ▷ Transpose for computational efficiency
- 5: $X_1 \leftarrow X_1^\top$
- 6: **for** $i = 1$ **to** k_{ns} **do**
- 7: $A_i \leftarrow X_i^\top X_i$ ▷ Gram matrix computation
- 8: $B_i \leftarrow bA_i + cA_i^2$
- 9: $X_{i+1} \leftarrow aX_i + X_iB_i$ ▷ Newton-Schulz update step
- 10: **if** $m > n$ **then** ▷ Restore original orientation
- 11: $X_{k_{ns}+1} \leftarrow X_{k_{ns}+1}^\top$
- 12: **return** $X_{k_{ns}+1}$

Algorithm 3 PowerIter: Approximate Largest Singular Value and Left Singular Vector

Require: Matrix $W \in \mathbb{R}^{p \times q}$, initial vector $u \in \mathbb{R}^p$ (normalized), iterations k (default: 1)

- 1: $u \leftarrow u / \|u\|_2$ ▷ Ensure initial normalization
- 2: **for** $i = 1, \dots, k$ **do**
- 3: $v \leftarrow W^\top u$
- 4: $v \leftarrow v / \|v\|_2$ ▷ Right vector (normalized)
- 5: $u \leftarrow Wv$
- 6: $u \leftarrow u / \|u\|_2$ ▷ Left vector (normalized)
- 7: $\sigma \leftarrow u^\top Wv$ ▷ Rayleigh quotient approximation of $\sigma_{\max}$
- 8: **return** σ, u

B. Ablations

We systematically evaluate the contribution of each component in our method using a Factorized Llama-94M model.

B.1. Effect of Orthogonalization and Spectral Renormalization

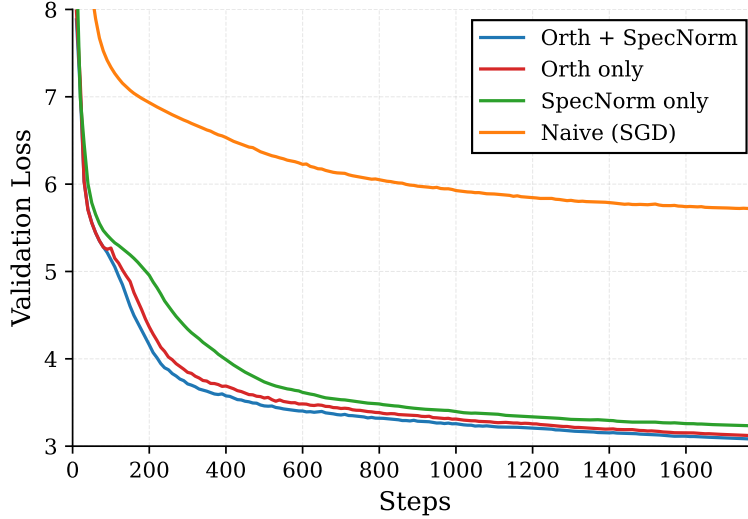


Figure 10. Ablation Study of Orthogonalization and Spectral Renormalization Components. Validation loss curves over 1,800 training steps for a Factorized Llama-94M model trained with different combinations of our method’s core components: combined orthogonalization and spectral renormalization (blue), orthogonalization only (red), spectral renormalization only (green), and naive SGD baseline (orange). While both individual components substantially improve upon the baseline, their combination (blue) achieves the lowest final validation loss (3.00), demonstrating that gradient orthogonalization and spectral renormalization provide complementary effects essential for stable low-rank training. The naive baseline exhibits severely impaired convergence, plateauing at 7.0 validation loss, highlighting the critical importance of spectral control in factorized architectures.

Gradient Orthogonalization	Spectral Renormalization	Perplexity	Val loss
×	×	1042.02	6.95
×	✓	24.01	3.18
✓	×	20.95	3.04
✓	✓	20.26	3.00

Table 2. Ablation Analysis of Gradient Orthogonalization and Spectral Renormalization Components. Systematic evaluation of each component’s contribution to final model performance on Factorized Llama-94M. Gradient orthogonalization (Orth) ensures updates lie in the constraint radius, while spectral renormalization (SpecNorm) controls the constraint radius based on the spectral norms of the factorized weight matrices. The naive baseline (neither component) achieves 1042 perplexity and 6.95 validation loss. Activating spectral renormalization alone reduces val loss to 3.18 (54% improvement), while orthogonalization alone achieves 3.04 (56% improvement). The full method combining both components yields optimal performance at 3.00 validation loss, demonstrating that these mechanisms provide synergistic control.

Figure 10 and Table 2 present an analysis of our two core components. The naive baseline fails catastrophically, while both components independently recover stable training. Combining orthogonalization and spectral renormalization achieves the best performance, confirming their complementary roles. Orthogonalization only benchmark yields the Muon (Jordan et al., 2024) optimizer.

B.2. Effect of Rank Ratio

Figure 11 and Table 3 examine sensitivity to the rank ratio. We defined rank ratio as the multiple used to set the low rank r using the input dimension n of a matrix sized $m \times n$. Rank ratios of $0.25 \times n$ and $0.4 \times n$ achieve comparable performance with 0.4 being slightly better. Aggressive compression to $0.125 \times n$ substantially degrades learning. We adopt 0.25 for all experiments to balance parameter efficiency with performance.

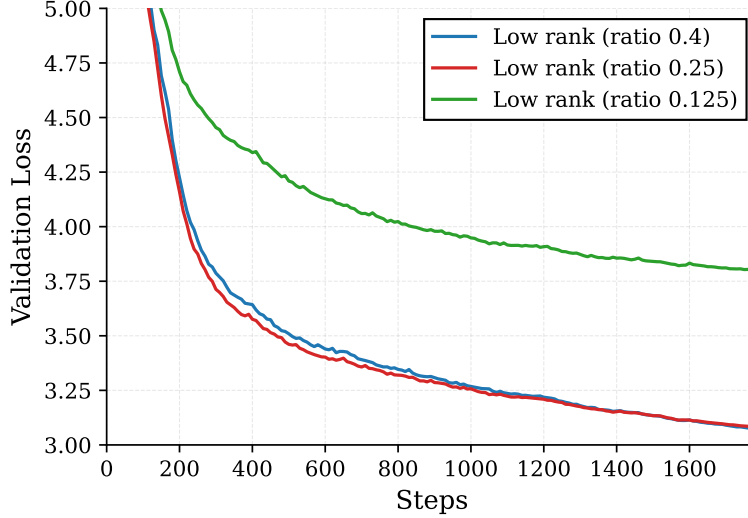


Figure 11. Effect of Rank Ratio in Low-Rank Factorized Training. Validation loss curves across 1,800 optimization steps for Factorized Llama-94M trained with different rank ratios under our method. Models with rank ratios 0.4 (blue) and 0.25 (red) exhibit nearly identical convergence dynamics with 0.4 being slightly better. In contrast, aggressive compression to rank ratio 0.125 (green) results in fundamentally degraded learning dynamics. We choose 0.25 for all our experiments as a best trade off.

In addition, Table 4 reports validation loss across optimization methods and rank ratios. We make two key observations: (a) Spectron consistently outperforms all low-rank baselines across every rank ratio, and (b) Spectron surpasses dense full-rank training at higher ranks. Performance continues to improve with rank as show in Figure 13.

B.3. Effect of Learning Rate on Training Stability

Training low-rank factorized models presents a critical challenge: extreme sensitivity to learning rate selection. Figure 12 demonstrates how this instability manifests across different optimization approaches and learning rates. This behavior aligns with our analysis in Section 4, where we show that large spectral norm fluctuations cause proportional changes in activation RMS, destabilizing training.

We observe that naive AdamW training fails completely at $\eta = 0.01$, producing loss spikes that lead to divergence. Large activation changes during gradient updates cause this failure. While reducing the learning rate to $\eta = 0.001$ achieves convergence, training proceeds slowly and inefficiently.

Self-guided training (Wei et al., 2024a) exhibits mixed results depending on the optimizer choice. When combined with Muon (Jordan et al., 2024)—an optimizer that applies gradient orthogonalization—self-guided training supports higher learning rates but produces inferior final performance. Self-guided training with AdamW (Kingma & Ba, 2015) at lower learning rates achieves better results, motivating our choice of AdamW (Kingma & Ba, 2015) for self-guided baselines in the main paper.

Our method combines orthogonalization with spectral renormalization to achieve both stable and fast convergence. At

Rank Ratio	Perplexity	Val loss
0.25	20.26	3.00
0.4	20.00	2.99
0.125	42.65	3.75

Table 3. Sensitivity of Low-Rank Training Performance to Rank Ratio Selection. Validation loss and perplexity as a function of rank ratio (proportion of retained dimensions relative to full-rank parameterization) for Factorized Llama-94M with our method. The rank ratio of 0.4 achieves optimal performance, marginally outperforming 0.25. However, aggressive compression to 0.125 results in substantial degradation. We choose a rank ratio of 0.25 for all our experiments to balance parameter reduction and performance tradeoff.

Stabilizing Native Low-Rank LLM Pretraining

Method	$r = 0.25$	$r = 0.4$	$r = 0.5$
Dense (Full-Rank)	3.070	3.070	3.070
Vanilla AdamW	3.274	3.196	3.171
Self-Guided	3.240	3.207	3.187
Spectron (Ours)	3.084	3.028	3.013

Table 4. Validation Loss Across Rank Ratios. Validation loss for Factorized Llama-94M trained with different low-rank optimization methods at rank ratios $r \in \{0.25, 0.4, 0.5\}$. Dense full-rank training (3.070) is shown as a reference. Spectron consistently achieves the lowest loss among all low-rank methods, and surpasses the dense baseline at $r = 0.4$ and $r = 0.5$.

$\eta = 0.01$, our approach converges smoothly without the instabilities that plague naive factorized training, while maintaining computational efficiency with only 1% FLOPs overhead. This demonstrates that explicit spectral control enables factorized models to leverage aggressive learning rates that would otherwise cause training failure.

B.4. Effect of factorizing fully connected layers only

While our main experiments apply low-rank factorization to all non-embedding matrices, we conduct an ablation study to evaluate Spectron’s effectiveness when restricting factorization exclusively to feedforward network (FFN) layers—the configuration adopted by Wei et al. (2024a). This controlled comparison directly addresses whether Spectron’s advantages persist under the more constrained setting proposed by prior work.

Figure 14 demonstrates that Spectron consistently outperforms both self-guided learning and naive AdamW training even when limiting low-rank factorization to FFN layers only. Spectron achieves a superior convergence properties within the exact architectural constraints proposed by Wei et al. (2024a). This result validates that Spectron’s optimization strategy provides tangible benefits beyond simply expanding the scope of factorization, and confirms its effectiveness across different factorization schemes.

C. Self-Guided Training

The self-guided training technique addresses optimization challenges in low-rank factorized weight matrices by introducing an auxiliary dense parameterization that facilitates stable training dynamics (Wei et al., 2024a).

The method computes the layer output as a weighted combination:

$$\mathbf{o} = \alpha \cdot W\mathbf{x} + (1 - \alpha) \cdot A(B\mathbf{x}), \quad (17)$$

where α follows a cosine decay schedule from 1 to 0 throughout training.

Initially, the dense matrix W dominates the computation, providing stable gradient signals that circumvent pathological landscapes inherent to factorized structures. As α decreases, the model progressively transitions to the factorized representation (A, B) , which inherits feature specialization learned by W through backpropagated gradients.

The dense matrix is initialized as

$$W_0 = A_0 B_0, \quad (18)$$

ensuring no behavioral change at the start and enabling application at any training stage, including fine-tuning.

To reduce computational overhead, a stochastic variant employs probabilistic sampling:

$$\mathbf{o} = \begin{cases} \alpha \cdot W\mathbf{x} + (1 - \alpha) \cdot A(B\mathbf{x}) & \text{with probability } p < \alpha, \\ A(B\mathbf{x}) & \text{otherwise,} \end{cases} \quad (19)$$

where $p \sim \text{Uniform}(0, 1)$ is sampled independently each forward pass.

This formulation reduces expected computational cost to approximately half the deterministic version while maintaining comparable accuracy. Applying this technique during half of training is said to increase total floating-point operations by 25% relative to using solely the factorized parameterization (Wei et al., 2024a).

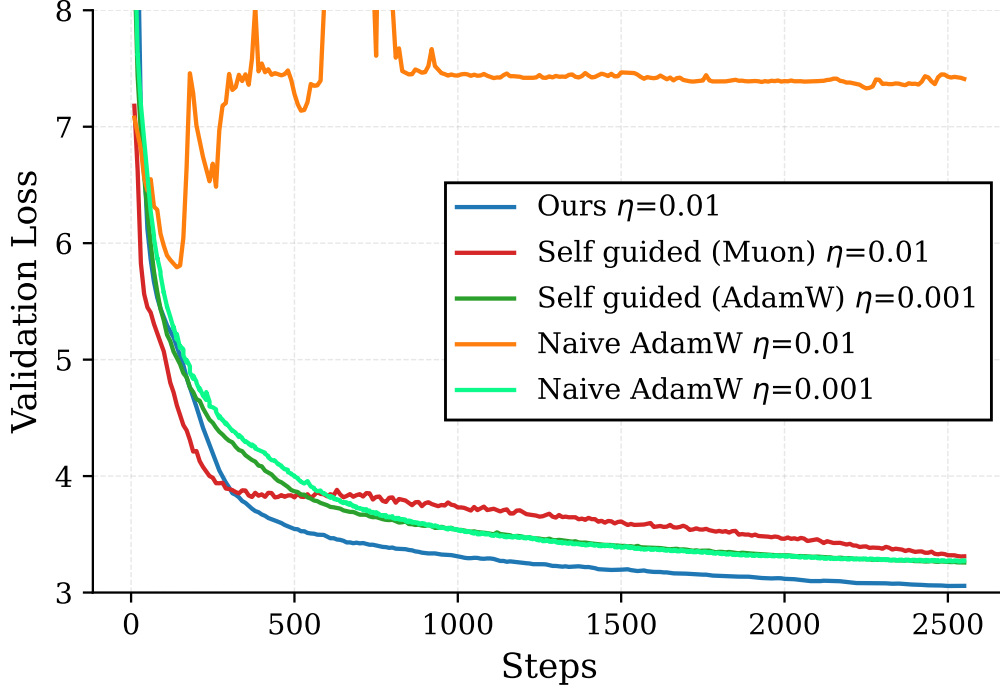


Figure 12. Higher Learning Rates Destabilize Factorized Training in Naive training. Validation loss curves comparing different optimization methods for low-rank Factorized Llama-94M across learning rates $\eta \in \{0.001, 0.01\}$. Naive AdamW (orange) diverges catastrophically at $\eta = 0.01$ and converges slowly at $\eta = 0.001$, demonstrating that standard optimizers cannot support aggressive learning rates for factorized models. Self-guided training (with AdamW (Kingma & Ba, 2015)) improves stability through dense matrix guidance but still requires 25% additional FLOPs and memory for auxiliary weights. We see little improvement on Self-guided training when using Muon (Jordan et al., 2024) optimizer. Our method (blue) achieves stable, fast convergence at $\eta = 0.01$ with only 1% FLOPs overhead via spectral renormalization and orthogonalization, eliminating the training instability inherent to factorized matrices.

D. Scaling Laws: Parametric Function Approach

We complement our IsoFLOP analysis (Section 6) with the parametric approach of Hoffmann et al. (2022). This method directly models validation loss as a function of model size N and training tokens D using the classical risk decomposition:

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad (20)$$

where E represents irreducible loss, and the power-law terms capture underfitting due to limited model capacity and limited data, respectively. We refer to Appendix D.2 of Hoffmann et al. (2022) for further details of this functional form.

Fitting Procedure. We fit this functional form to all 39 data points from our IsoFLOP experiments, spanning model sizes from 47M to 1.5B parameters. We minimize the Huber loss (Huber, 1992) with $\delta = 10^{-3}$ between predicted and observed log loss using scipy (Virtanen et al., 2020) L-BFGS-B (Nocedal, 1980) algorithm, obtaining the following estimates:

$$A = 1000, \quad \alpha = 0.398, \quad (21)$$

$$B = 1000, \quad \beta = 0.332, \quad (22)$$

$$E = 1.777. \quad (23)$$

Compute-Optimal Allocation. Following Hoffmann et al. (2022), we derive the compute-optimal model size and token count by minimizing $L(N, D)$ subject to the constraint $C = 6ND$. Here C denotes the total compute budget. This yields:

$$N_{\text{opt}}(C) \propto C^{\frac{\beta}{\alpha+\beta}} = C^{0.45}, \quad D_{\text{opt}}(C) \propto C^{\frac{\alpha}{\alpha+\beta}} = C^{0.55}. \quad (24)$$

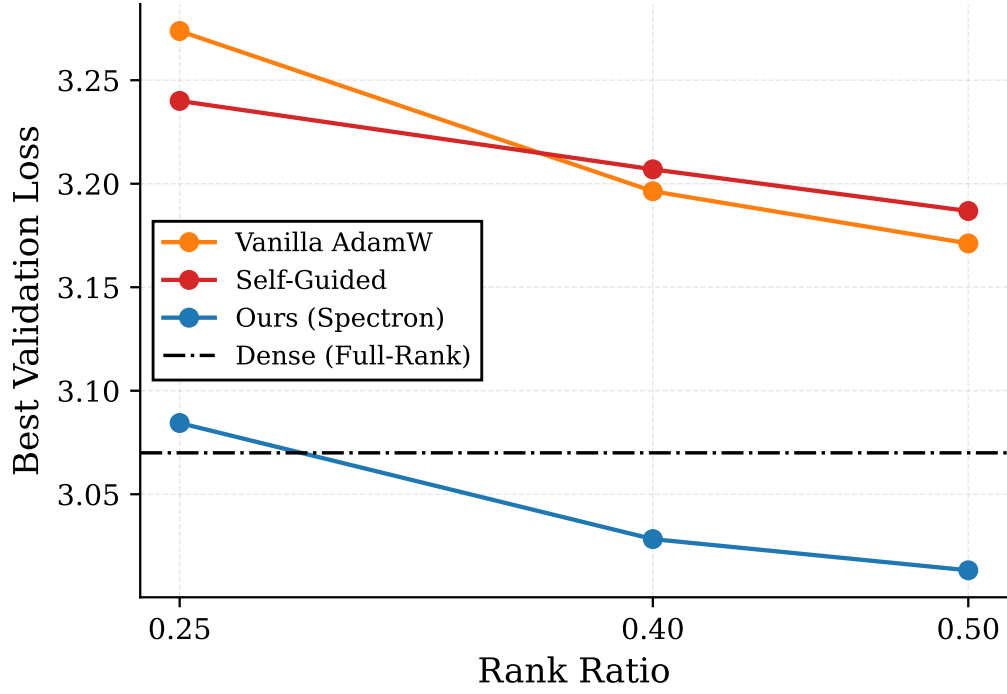


Figure 13. Best Validation Loss Across Rank Ratios Best validation loss for Factorized Llama-94M trained at rank ratios $r \in \{0.25, 0.4, 0.5\}$ under three low-rank optimization methods—Vanilla AdamW (orange), Self-Guided (red), and our method Spectron (blue)—with dense full-rank training shown as a reference (black dash-dot line). Spectron consistently achieves substantially lower loss than both low-rank baselines at every rank ratio, and crosses below the dense full-rank reference at $r \approx 0.26$, surpassing it for all higher ranks.

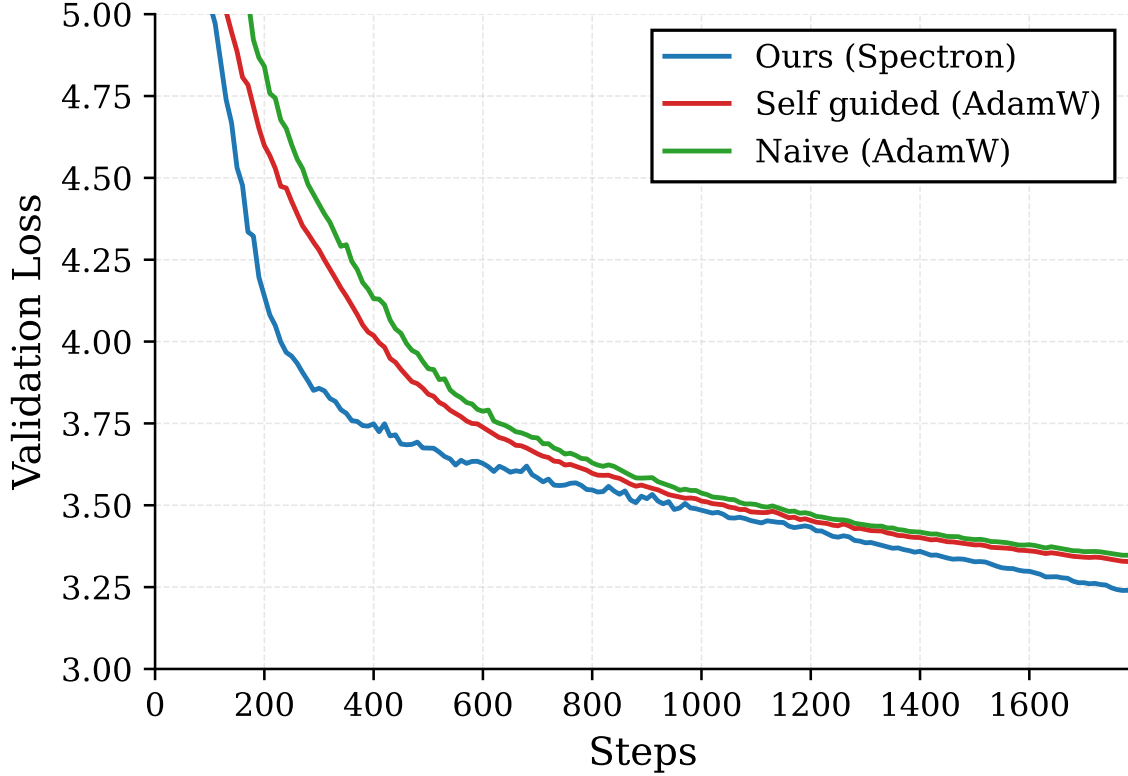


Figure 14. **Spectron Outperforms Baselines when only fully connected layers factorized to low rank** Validation loss curves comparing Spectron (blue) against self-guided learning (red) and naive AdamW training (green) when applying low-rank factorization exclusively to feedforward layers. Spectron achieves lower validation loss throughout training and converges faster, surpassing self-guided learning even within the setting proposed by Wei et al. (2024a)

These exponents closely match the Chinchilla scaling laws ($N_{\text{opt}} \propto C^{0.46}$, $D_{\text{opt}} \propto C^{0.54}$), but are consistently slightly smaller for optimal parameter size and larger for optimal token count. This indicates that, for a fixed compute budget, compute-optimal low-rank pretraining favors smaller models trained on more tokens compared to dense training. The consistency across two independent fitting approaches (IsoFLOP in Section 6 and parametric) strengthens confidence in this scaling characterization.

E. Implementation details

We describe our experimental setup and implementation choices in this section. All models follow the Llama-3 architecture with RoPE (Su et al., 2024) positional embeddings. We use the LLama-2 (7B) tokenizer with a vocabulary size of 32,000.

E.1. Model Configurations

For our main experiments, we evaluate three model scales: Llama-134M, Llama-500M, and Llama-780M. For low-rank variants, we apply a rank ratio of 0.25, reducing the effective rank to $0.25 \times n$ where n is the input dimension. Table 5 summarizes the architectural configurations for these models.

E.2. Scaling Law Experiments

For scaling law analysis, we use base models ranging from 60M to 2.7B parameters. We apply the same rank ratio of 0.25 to create low-rank variants, yielding reduced models from 47M to 1.5B parameters. Table 6 provides the complete architectural details for all model scales.

Table 5. Main model configurations used in our experiments.

Model	Hidden Size	Layers	Heads	Parameters
Llama-134M	768	12	12	134M
Llama-500M	1280	20	20	500M
Llama-780M	1536	24	24	780M

Table 6. Model configurations for scaling law experiments. All models use the same number of key-value heads as query heads.

Base Params	Hidden Size	Layers	Heads	Low-Rank Params
60M	512	8	8	47M
92M	640	10	10	68M
134M	768	12	12	94M
150M	768	14	12	101M
220M	896	16	14	143M
325M	1024	20	16	200M
500M	1280	20	20	297M
780M	1536	24	24	454M
835M	1536	26	24	484M
1.4B	2048	24	16	780M
1.7B	2048	30	16	943M
2.7B	2560	32	20	1.5B

E.3. Training Setup

We use cosine (Loshchilov & Hutter, 2016) scheduler with cycle length set to total training steps and first 5% steps as warmup. We decay the learning rate to 0. We conduct hyperparameter sweeps across learning rates and weight decay values to ensure optimal performance. Learning rates are swept across three logarithmic scales (10^{-1} , 10^{-2} , 10^{-3}) and three linear multipliers ($1\times$, $5\times$, $7\times$ of each log scale) for all configurations. Weight decay (Loshchilov & Hutter, 2019) is swept from 10^{-1} to 10^{-3} . We report the best results obtained from these sweeps for all experiments. When AdamW (Kingma & Ba, 2015) is used we use β_1 and β_2 as 0.9 and 0.95. For Muon (Jordan et al., 2024) experiments we use 0.95 for momentum.

Following Wei et al. (2024a), we initialize low-rank models using spectral initialization (Khodak et al., 2021) while applying standard decoupled weight decay regularization (Loshchilov & Hutter, 2019). All models are trained on the Fineweb (Penedo et al., 2024) corpus with a batch size of 512 sequences and a sequence length of 2048 tokens, yielding $512 \times 2048 = 1,048,576$ tokens per training step. We train the dense models Llama-134M, Llama-500M, and Llama-780M for 1786, 5664, and 8657 steps, respectively. For the factorized models, Factorized Llama-94M, Factorized Llama-297M, and Factorized Llama-454M, we train for 2556, 9537, and 14878 steps, respectively. All experiments are conducted on NVIDIA H100 GPUs with mixed precision using bf16.

F. Wall-Clock Time Analysis

Table 7. Wall-clock training times (in seconds) for different low-rank optimization methods across model scales.

Method	Factorized Llama 94M (s)	Factorized Llama 297M (s)	Factorized Llama 454M (s)
Naive Low-rank (AdamW)	1,965.82	17,355.04	36,677.68
Self-Guided	2,369.59	18,525.23	39,067.47
Spectron (Ours)	2,013.39	17,700.99	36,861.37

We compared this with multinode training on 2 nodes having 4 H100s each interconnected by infiniband. Spectron adds only $\sim 2\text{--}2.4\%$ wall-clock time over naive low-rank AdamW, confirming the Newton-Schulz iterations and power-method estimates are lightweight in practice and it is substantially less than Self-Guided ($\sim 6.5\%$) (Wei et al., 2024a).

The wall-clock times for dense training are 1,473.61 s (Llama 134M), 15,233.55 s (Llama 500M), and 30,974.24 s (Llama 780M). The gap between dense and low-rank times is due to dense training using highly optimized kernels while our low-rank forward pass uses two unfused matmuls with launch overhead that outweighs FLOP savings at this scale; in

addition, the FLOP-matched models train on more tokens and our data loading pipeline has not been specifically optimised. Custom fused kernels can close this gap.

G. Comparison with recent Low rank training methods

In this section we compare with recent low rank training methods [Mo et al. \(2025\)](#) and [Liu et al. \(2025b\)](#). We want to highlight an important distinction: CoLA modifies the model architecture by replacing weight matrices with autoencoder modules for low-rank activations, whereas Spectron and LoRO operate within the standard architecture. Despite this difference, we compare against both for completeness on a LLaMA-94M model.

Method	Arch. Change	Perplexity	Wall Clock (s)
LoRO	No	24.56	2,212
Spectron	No	21.86	2,013
CoLA	Yes	24.21	2,479
CoLA + Spectron	Yes	19.95	2,594

Table 8. Performance comparison of LoRO, Spectron, CoLA, and their combination on the LLaMA-94M model.

Spectron outperforms LoRO by 2.7 perplexity points while also being $\sim 10\%$ faster in wall-clock time. Furthermore, Spectron is complementary to architectural approaches like CoLA so combining the two yields the best result overall (19.95), demonstrating that our optimizer-level contribution is orthogonal to architecture-level low-rank methods and can be composed with them for additional gains.

G.1. Extension of Spectron to Nonlinear Factorizations

We show below how we extend Spectron to the nonlinear factorization used in CoLA ([Liu et al., 2025b](#)).

When a nonlinearity (here we consider ReLU ([Nair & Hinton, 2010](#))) is inserted between the factors, the effective mapping becomes $W_{\text{eff}} = A\sigma(Bx)$. For ReLU (and element-wise gates like SiLU ([Elfwing et al., 2018](#)) in the approximate sense), this can be written as $W_{\text{eff}} = ADB$ where $D = \text{diag}(\mathbf{1}[Bx > 0])$ is a data-dependent binary diagonal mask. Since D is a binary diagonal matrix, $|D|_2 \leq 1$ always. The perturbation bound becomes

$$|\Delta W_{\text{eff}}|_2 \leq |\Delta A|_2 |D|_2 |B|_2 + |A|_2 |D|_2 |\Delta B|_2 + |\Delta A|_2 |D|_2 |\Delta B|_2. \quad (25)$$

Using $|D|_2 \leq 1$, this reduces to exactly the same bound as the linear case shown in the main paper:

$$|\Delta W_{\text{eff}}|_2 \leq |\Delta A|_2 |B|_2 + |A|_2 |\Delta B|_2 + |\Delta A|_2 |\Delta B|_2. \quad (26)$$

The activation mask drops out entirely. Consequently, the spectral LR scaling rule $\eta/(|A|_2 + |B|_2 + 1)$ derived for the linear factorization applies to CoLA’s nonlinear factorization with the same structural guarantees if the non-linearity is ReLU. This achieves the same goal of controlling the spectral norm growth of the product W without destabilizing the activations after applying W to x .

G.2. Comparison with ReLoRA and SST

In this section we compare our method with full-rank training methods which use low rank updates. We note that this has some important differences in original setup of our paper. We train a low-rank model from scratch without any auxiliary weights, whereas ReLoRA ([Lialin et al., 2024](#)) requires a full-rank warm start and periodical merges using auxiliary weights, and SST ([Zhao et al., 2025](#)) uses a spectral parameterization that decouples singular values from singular vectors and keeps the $U\Sigma V$ parameterization in memory.

We ran ReLoRA ([Lialin et al., 2024](#)) and SST ([Zhao et al., 2025](#)) both with and without Spectron. Our experiments show that Spectron’s normalization provides complementary benefits even when combined with ReLoRA and SST.

The results show that Spectron delivers a 12.9% perplexity reduction on ReLoRA and a 20.4% reduction on SST, demonstrating that our spectral-norm stabilization is complementary to both dynamic-rank and spectral-parameterization approaches. These auxiliary experiments strengthen the paper by situating Spectron’s contribution relative to alternative training strategies.

Table 9. Perplexity comparison of Spectron against ReLoRA (Lialin et al., 2024) and SST (Zhao et al., 2025) on a 134M base model (94M trainable parameters).

Method	Warm start	Base params	Total params (in memory)	Trainable params	Perplexity	Reduction (%)
Spectron (Our setup)	No	134M	94M	94M	21.86	—
ReLoRA	Yes	134M	162.14M	94M	26.06	—
ReLoRA + Spectron	Yes	134M	162.14M	94M	22.70	12.90
SST	No	134M	175.54M	94M	29.40	—
SST + Spectron	No	134M	175.54M	94M	23.40	20.40

H. Spectral norm variation across layers

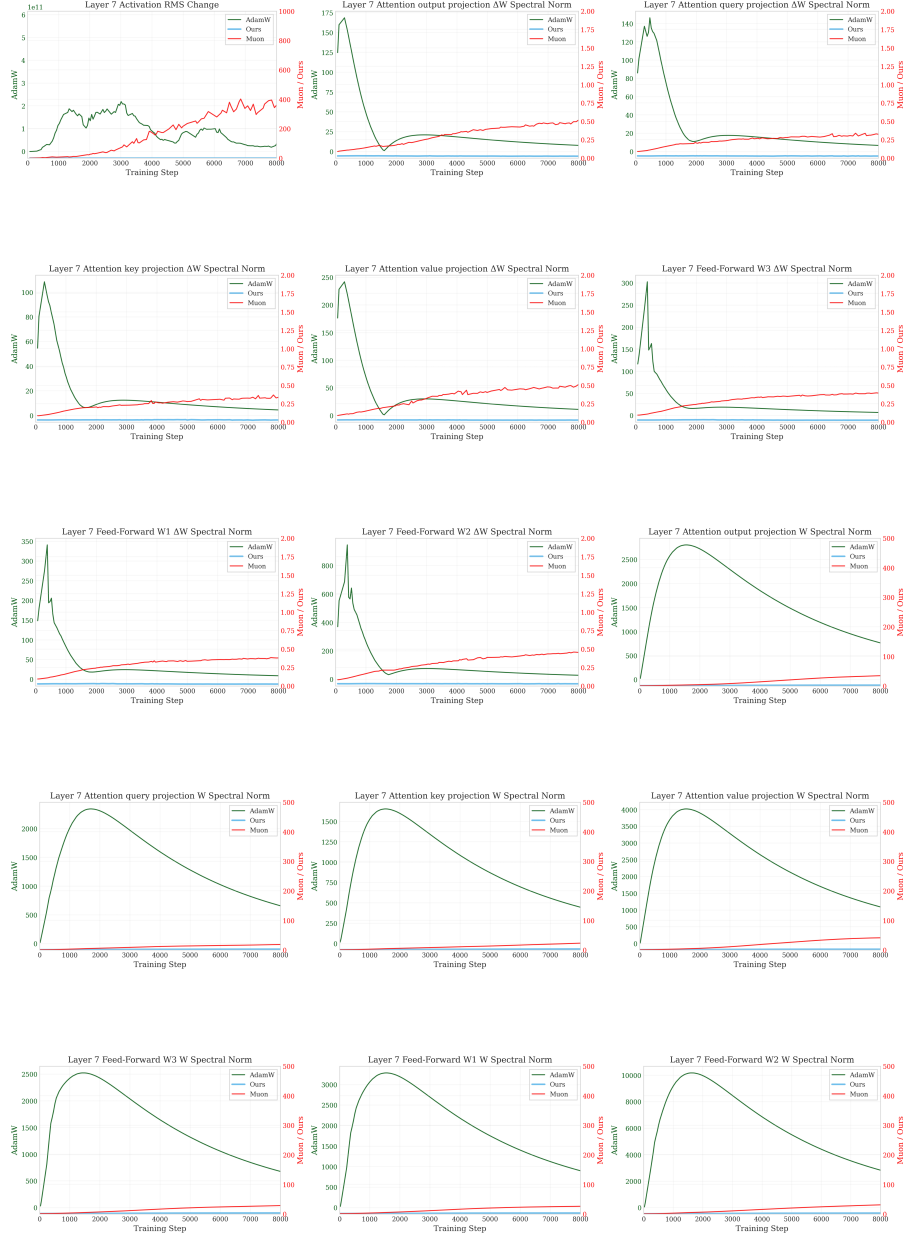
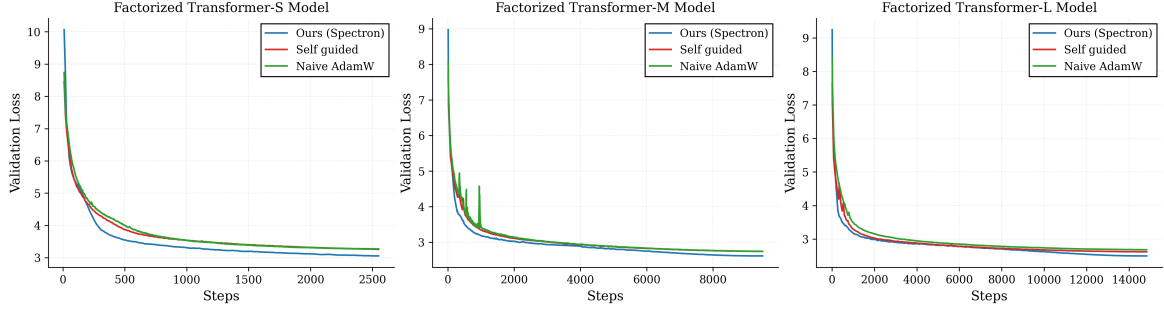
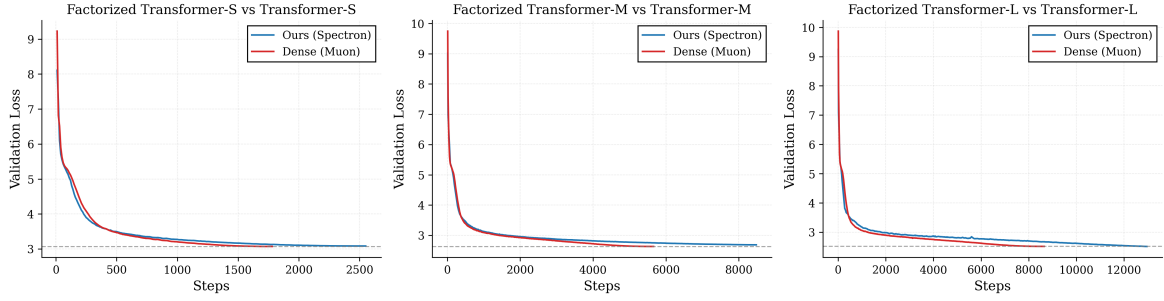


Figure 15. Spectral norm variation across layers

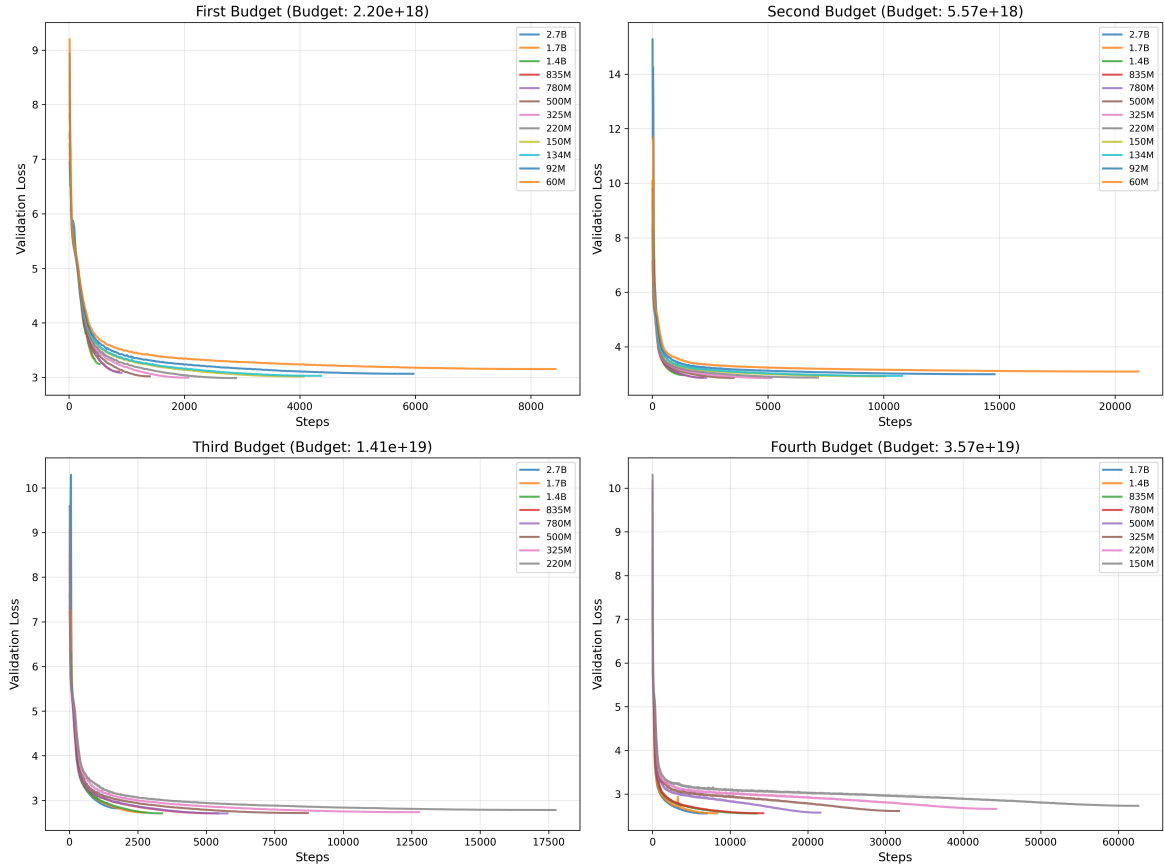
I. Training curves



(a) Training curves of our baseline comparisons in Section 5



(b) Training curves of our comparisons with dense models in Section 5



(c) Training curves of our optimal model allocation experiments in Section 6

Figure 16. Training curves across different experimental settings